# Algorithms Portfolio

*Curated algorithmic solutions with correctness and complexity analysis*

---

**Ryan McMillan**

MCompSci (cand.) – University of New England

C++ primary • Python secondary

January 18, 2026

## License

# Glossary

**Amortized analysis**
: An analysis technique that considers the average cost of operations over a sequence of operations, even if individual operations may be expensive.

**Invariant**
: A property that remains true before and after each iteration of an algorithm or loop.

**In-place algorithm**
: An algorithm that transforms its input using only a constant amount of additional memory.

**Loop invariant**
: An invariant associated with a loop, used to prove correctness via initialization, maintenance, and termination.

**Time complexity**
: An asymptotic measure of how an algorithm's running time grows as a function of input size.

# Contents

**Glossary** **2**

**1 Introduction** **4**
1.1 How to Use This Book . . . . . 4
1.2 Motivation . . . . . 4
1.3 Acknowledgements . . . . . 4

**2 Arrays & Sequences** **5**
2.1 Core Ideas . . . . . 5
2.2 Common Pitfalls . . . . . 5
2.3 Easy . . . . . 5
2.3.1 Plus One . . . . . 5

# 1 Introduction

This document is a curated portfolio of algorithmic solutions. Each selected problem includes:

- clear problem restatements,
- key invariants / observations,
- pseudocode,
- correctness sketches,
- complexity analysis,
- C++ (primary) and Python (secondary) implementations.

The focus is on clarity, correctness, and reasoning.

## 1.1 How to Use This Book

Problems include an optional **Author's Thinking** block, the authors plain language thinking and problem framing, which is excluded from exemplar builds. Use this book to understand how to formally reason with and structure algorithm and data structures problems.

## 1.2 Motivation

This portfolio was created as a structured, self-directed study of fundamental algorithms and data structures. While my formal coursework emphasizes applied computer science and systems-level engineering, it does not include a dedicated course focused on algorithmic problem-solving in the style commonly expected in technical interviews and quantitative software roles.

To address this, I undertook a systematic review of core algorithmic techniques, organizing problems by data structure and algorithmic paradigm, and documenting each solution with an explicit algorithm description, correctness argument, and complexity analysis. The goal is not only to arrive at correct implementations, but to practice communicating algorithmic reasoning clearly and rigorously.

This document serves both as a personal reference and as evidence of deliberate practice in algorithmic reasoning beyond formal coursework.

Many of the problems included here are sourced from commonly used interview problem sets, and were selected to emphasize invariant-based reasoning, edge case analysis, and asymptotic performance trade-offs.

Informal reasoning notes are included selectively to document thought process and learning progression; these sections are omitted in derived exemplar or presentation-focused versions of this material.

## 1.3 Acknowledgements

This work was informed by a number of open educational resources that emphasize rigorous algorithmic reasoning and clear exposition. In particular, the structure and approach to correctness arguments and asymptotic analysis were influenced by *MIT OpenCourseWare 6.006: Introduction to Algorithms.*

Additional reference was made to publicly available problem discussions, editorials, and textbooks where appropriate. All implementations, explanations, and written analyses in this document are my own, and any external influences are acknowledged at a conceptual level rather than reproduced directly.

# 2 Arrays & Sequences

Arrays are the foundational data structure in most algorithmic problems. They provide constant-time indexed access and strong cache locality, but require careful reasoning about indices, bounds, and in-place mutation.

This chapter focuses on recognizing array-based problem patterns and maintaining clear invariants over contiguous memory.

## 2.1 Core Ideas

- Index-as-state reasoning
- In-place mutation vs auxiliary storage
- Prefix and suffix accumulation
- Simulation of elementary operations (e.g. addition, carry propagation)

## 2.2 Common Pitfalls

- Off-by-one errors at boundaries
- Unintended data overwrites during in-place updates
- Assuming sorting or uniqueness without verifying constraints
- Over-allocating space when a single pass suffices

## 2.3 Easy

The following problems introduce fundamental array manipulation techniques. The emphasis is on establishing and maintaining simple invariants while iterating through the array.

### 2.3.1 Plus One

**Difficulty:** Easy **Tags:** Arrays, Simulation **ID:** LC-066

**Problem.** Given a non-empty array of digits representing a non-negative integer, increment the integer by one and return the resulting array of digits.

The most significant digit is stored at the beginning of the array, and each digit is in the range $[0, 9]$.

**Key idea.** Incrementing the number requires simulating elementary addition from right to left. Carry propagation only affects trailing digits equal to nine; once a digit strictly less than nine is incremented, the process terminates.

**Author's Thinking.**

> My initial instinct was to convert the digit array into an integer, add one, and convert it back. This approach fails due to potential overflow and violates the problem constraints.
>
> What simplified the reasoning was treating the operation as grade-school addition and maintaining the invariant that all digits to the right of the current index are already correct after carry handling.

**Algorithm.**

```
for i from n - 1 downto 0:
    if digits[i] < 9:
        digits[i] = digits[i] + 1
        return digits
    digits[i] = 0

prepend 1 to digits
return digits
```

Listing 1: Pseudocode

**Correctness sketch.** We iterate from the least significant digit toward the most significant digit. At each iteration, the invariant maintained is that all digits to the right of the current index represent the correct result after increment and carry propagation.

If a digit less than nine is encountered, incrementing it produces the correct result and no further carry is required. If all digits are nine, each is set to zero and a leading one is prepended, yielding the correct incremented number. Thus, the algorithm always returns the correct result.
**Complexity.** The algorithm runs in $O(n)$ time, where $n$ is the number of digits. It uses $O(1)$ additional space beyond the output array.

### C++ Implementation

```cpp
/*
* LC-066 - Plus One
*
*/

#include <iostream>

class Solution {
public:
    vector<int> plusOne(vector<int>& digits) {
        for(int i = digits.size(); i-- > 0 ; ) {
            if (digits[i] != 9) {
                digits[i]++;
                return digits;
            }
            else {
                digits[i] = 0;
            }
        }

        vector<int> newDigits(digits.size() + 1);
        newDigits[0] = 1;
        for (int j = 1; j < newDigits.size(); j++) {
            newDigits[j] = digits[j-1];
        }

        return newDigits;
    }
};
```

Listing 2: C++ Implementation

### Python Implementation

```python
# LC-066 Plus One

```

```python
from typing import List

def plusOne(digits: List[int]) -> List[int]:
    for i in range(len(digits) - 1, -1, -1):
        if digits[i] < 9:
            digits[i] += 1
            return digits
        digits[i] = 0

    return [1] + digits
```

Listing 3: Python Implementation